trade off with the resources employed, and practical algorithms manage to keep this probability small while using a reasonable amount of computing power and memory.

For decision problems, when the answer of the *Monte Carlo* algorithm is just `true`/`false`, we have three possible situations:

- The algorithm is always correct when it returns false (so-called *false-biased* algorithm).
- The algorithm is always correct when it returns true (so-called *true-biased* algorithm).
- The algorithm might return the wrong answer indifferently for the two cases.

While Monte Carlo algorithms are usually not deterministic in the amount of resources needed, they are used often as the dual of a Las Vegas algorithm. Suppose we have an algorithm A that always return the correct solution, but whose resource consumption is not deterministic. If we have a limited amount of resources, we can run A till it outputs a solution or exhausts the allotted resources. For instance, we could stop randomized quicksort after n*log n swaps.

This way, we are trading accuracy (the guarantee of a correct result) for the certainty to have a (sub-optimal) answer within a certain time (or using at most a certain amount of space).

It's also worth noting that, as we mentioned, for some randomized algorithms we don't even know if they will eventually stop and find a solution (although this is usually